

Technology flexibility as enabler of robust application development in community source: The case of Kuali and Sakai

Manlu Liu^{a,*}, Harry Jiannan Wang^b, J. Leon Zhao^c

^a Rochester Institute of Technology, United States

^b University of Delaware, United States

^c City University of Hong Kong, China

ARTICLE INFO

Article history:

Received 5 June 2010

Received in revised form 4 June 2012

Accepted 14 June 2012

Available online 20 July 2012

Keywords:

Technology flexibility

Open source

Community source

Workflow technology

Service oriented architecture

Agile enterprise

ABSTRACT

Technology flexibility has been an important topic in software engineering since the start of computerized business applications, which require frequent changes to system specifications due to ever changing business requirements. Achieving a higher degree of technology flexibility has been a long-running challenge to software engineers and project managers. Recently, there has been a new software development approach called “community source” consisting of numerous development partners that are also users of the software. In community source, technology flexibility is even more important than usual due to the increase in complexity and uncertainty of software requirements by its many development partners in the community. In this paper, we investigate two community source cases, i.e., Kuali and Sakai, and examine how technology flexibility is achieved in application software engineering. The principles generated from this study should offer useful insights to the continuous efforts toward making more robust business applications in support of agile enterprises.

© 2012 Elsevier Inc. All rights reserved.

1. Introduction

Technology flexibility has been an important topic in software engineering since the start of computerized business applications, which require frequent changes to system requirements due to ever changing business requirements (Zeng and Zhao, 2002). In fact, we still have trouble describing system requirements precisely because of much uncertainty in business environments and user requirements. As such, software specifications and the associated code require frequent changes throughout their construction and usage. Achieving a higher degree of technology flexibility has been a long-running challenge to software engineers and project managers.

Recently, there has been a new software development approach called “community source” consisting of numerous development partners that are also users of the software. In community source, the community evolves over time, and so do the system requirements that depend on the collective application requirements by the development partners. Consequently, in community source, technology flexibility is even more important than usual due to increased uncertainty in software requirements from the community.

Community source has achieved initial success recently, as in the cases of Kuali (www.kuali.org) and Sakai (www.sakai.org), which raises the question of how community source has been dealing with the technology flexibility challenge. Understanding this new phenomenon should help alleviate this long-running challenge to the software industry. To this end, we investigate issues of technology flexibility in the context of community source since the extreme uncertainty in software requirements due to the evolving and complex nature of the developer community makes technology flexibility an intriguing phenomenon to study.

The structure of the paper is as follows. First, we review the recent research in technology flexibility, open source and community source development. The Community Source Model is introduced in Section 2. The related research methodology applied in this study and two real world of cases—Kuali and Sakai are introduced in Section 3. Next we explain how Kuali and Sakai achieve flexibility via various software techniques in Sections 4 and 5, respectively. Then we discuss our research findings by means of several principles stemming from the Kuali and Saikai cases observation, document analysis in Section 6. Finally, several conclusions are generalized from this study. Limitations and future research directions are discussed.

2. Literature review

In this section, we summarize the recent thinking in the related research areas, including technology flexibility, open source development, and community source development.

* Corresponding author.

E-mail addresses: manluliu@saunders.rit.edu (M. Liu), hjwang@lerner.udel.edu (H.J. Wang), jlzhao@cityu.edu.hk (J. Leon Zhao).

2.1. Technology flexibility

Flexibility has become a key competitive factor in many business sectors, such as manufacturing (Gupta and Somers, 1992; Ramasesh and Jayakumar, 1991), the high-tech industry (Evans, 1991), automation (Adler, 1998), finance (Mason, 1986), and information systems (Davenport and Linder, 1994). *Flexibility in management* is defined as the degree to which an organization possesses a variety of actual and potential procedures, and the rapidness with which it can implement these procedures to increase the control capability of the management and improve the controllability of the organization over its environment (De Leeuw and Volberda, 1996). Deiters et al. (2000) classify flexibility into four dimensions: process flexibility, inter-organizational flexibility, flexible management and knowledge, and flexible task allocation.

Technology flexibility is defined as “the ability to adapt to both incremental and revolutionary change in the business or business process with minimal penalty to current time, effort, cost, or performance” (Nelson et al., 1997). Nelson et al. (1997) proposed a framework to measure technology flexibility using factors including modularity, change acceptance, consistency in the structural flexibility dimension, and rate of response, expertise, and coordination of actions in the process flexibility dimension.

Companies have been using emerging technologies to enable flexibility in the enterprise computing environment. Technology flexibility in IT infrastructure, which has been studied by many researchers, enables an organization to develop new applications and modify existing applications more quickly and easily and is becoming a key to an organization's sustainable competitive advantage. Research efforts have focused on the impact of flexible technologies on organizations.

Duncan (1995) described technical IT infrastructure flexibility through the qualities of connectivity, compatibility, and modularity. Recent innovations in information technology, such as component-based and service-oriented software architectures (Bieberstein et al., 2006), Web services (Whiting, 2003), autonomous computing (Horn, 2001) and mobile applications (Keng et al., 2001), promise greater flexibility than the mainframe, client/server, and non-mobile system. Kettinger et al. (2010) shows how to design enterprise IT architecture by applying IT innovation to optimize flexibility so that the specific customers' demands can be met.

2.2. Open source/community source

Open source has become an important software engineering methodology in which the source code of a system is made available to its adopters (Ducheneanaut, 2005). However, open source is a very complex concept because of variations on how it is developed, marketed, and adopted. The most well-known example is the Linux system that is developed and maintained by a global network of volunteers and distributed by some commercial companies such as Red Hat (www.redhat.com). Many versions of Linux are distributed currently by these commercial companies who generate revenue by adding new features and providing consulting services. Many other open source systems are developed with government funding as output of a research project and offered online for free adoption with or without technical support (www.sf.net).

As a research domain, open source has received a lot of attention from various researchers, particularly regarding communication issues. Curtis et al. (1988) found that large projects have extensive communication and coordination needs that are not mitigated properly by documentation. These collaborative problems are exacerbated when the development team is geographically or organizationally distributed, as is becoming increasingly common. More effort is required for more systematic interactions when

participants are distant and unfamiliar with each other's work (Ocker and Fjermestad, 2000). Kraut and Streeter (1995) found that the formal and informal communication mechanisms for collaborative work on large-scale, complex software projects are important for sharing information and achieving collaboration in software development. Stewart and Gosain (2006) developed a framework of the main tenets of the open source software (OSS) ideology, which impacts team effectiveness in various ways. Recently, Fitzgerald (2006) found the field of open source has undergone a major transformation from its free software origins to a new form of community-based development and adoption.

As a unique form of open source, community source develops scalable and flexible information systems (Liu et al., 2007, 2010). Community-source application software for higher education appears to be the next arena for open source efforts (Wheeler, 2004; Wang et al., 2010). Wheeler (2007) argued that developing a collaborative capability is not an option; it is a necessity for effective college and IT organizations. Liu and Tu (2011) present various research directions of community source. In this paper, we focus on studying technology flexibility issues in community source, which are not well studied in the literature.

3. The community source model

Most recently, a new approach to enterprise application software development is community-based open source or “community source” (Liu et al., 2007), which aims to help institutions develop software collaboratively when they cannot find suitable commercial-off-the-shelf software. Collaborative software development is necessary because the overall cost of in-house development is much higher than that of buying commercial software (Perry and Quirk, 2007). Community source provides an alternate solution by applying strategic alliance to the development of open source software. In other words, community source enables organizations to pool their resources to develop shorable enterprise applications at a fraction of the costs that each individual institution would have to bear.

The community source approach is a unique form of open source software development (Liu et al., 2007). First, community source requires a formal collaboration among multiple institutions via a virtual organization. Second, many of the investments of developers' time, design, and project governance come from institutional contributions by colleges, universities, and some commercial firms rather than from individuals (Wheeler, 2007). Third, the community source project requires significant initial investments by the partner institutions, while an ordinary open source project might not. Some of these three features might appear in other types of open source projects, but a community source project requires all of them.

Community source is a complex endeavor because the community must balance the various, sometimes even conflicting, requirements from all development partners. As such, community source is more challenging than an ordinary open source project or commercial software for which variation in requirements is not a major issue. In our view, community source has more stringent requirements in flexibility in order to deal with complex requirement analysis and change management. Recently emerged technologies such as web services, service-oriented architectures, and workflow automation can help make a system more customizable. As we illustrate in the paper, community source takes notable advantage of these emerging technologies to enable more technology flexibility. As such, how technology flexibility is achieved through community source is a very interesting issue to study. In order to better understand community source, a comparison is

Table 1
Comparison among community source, ordinary open source and commercial software.

	Community source	Ordinary open source	Commercial software
Definition	The application development requires initial investments which are shared by the partners; the result is open source.	The software is free for adopters; the developers are volunteers.	The users need to buy the software; dedicated investment is required to develop the software.
Builders	Organized teams of formal employees from multiple partner institutions	Loosely connected volunteer developers	System development departments in software companies
Users	Development partners, deployment members, and others	Organizations or individuals who use the software for free	Organizations or individuals who pay for the software
Customization	The system is designed to meet the requirements of development partners	The system is designed to meet some generic requirements	Standard package
Technology flexibility requirements	High	Medium	Low

made among community source, ordinary open source and commercial software (Table 1).

4. Research methodology

We follow the case study method for this first-cut study on the issue of technology flexibility in community source (Eisenhardt, 1989; Yin, 1994). The data we collected are qualitative data from two real world cases—Kuali and Sakai. The propositions are observable in our case analysis.

The case study method is especially appropriate for contemporary phenomenon such as technology flexibility in community source, where theory and research are still in their formative stages. Yin (1994) suggests that whenever there are two cases, a cross-case analysis of the findings is likely to be more robust when it results in confirming, challenging, or extending the theory.

4.1. The Kuali case—an administrative system

Kuali is a consortium of universities that came together to develop an open source financial service system, starting with the conversion of the Indiana University's Financial Information System to the web in 2004 (www.kuali.org). The original motivation for the project was that existing financial systems used in the universities are outdated and too difficult to maintain. The commercial products are often too expensive and hard to customize; some institutions paid tens of millions of dollars to companies for software and installation, but still need to build 15% of all the features they need themselves to handle specific financial transaction requirements. Some schools and colleges find they must operate expensive "shadow systems" to provide needed features that are absent in currently available ERP packages. The alternatives to buying off-the-shelf packaged financial software are equally daunting and can be considered by only the largest institutions. The Kuali project provides an attractive alternative to the "buy or build" dilemma. It pools institutional resources to develop an open source financial system, thus dramatically reducing the cost of managing fiscal data and processes in higher education.

Indiana University and the University of Hawaii led the effort to build Kuali, which was launched in August 2004. In March 2005, the Kuali project got an award of \$2.5 M from the Andrew W. Mellon Foundation. In the same year, four new partners, Cornell University, San Joaquin Delta College, Michigan State University, and the University of Arizona, joined as core investors. In April 2006, the Kuali Project announced the availability of the Kuali Test Drive, which enables institutions to explore Kuali Financial Systems. In July 2006, the University of California joined Kuali. In October 2006, the Kuali

foundation released Phase I. Phase IIa and Phase IIb was released in July 2007 and in July 2008, respectively.

Kuali uses a community source approach to allow member universities to share the cost and benefits of system development. From Fig. 1, we can see that KFS is developed under a formal and complicated organization structure. The development partners work in a project organization. The Kuali Board is the final decision maker during the development of Kuali. The Kuali functional council and Kuali technical council take care of the functional issues and technical issues of Kuali, respectively. The extended board, Kuali investors, Kuali functional council, Kuali technical council and the project manager report to the Kuali board. The project manager supervises the project staff and development staff. We can also see that the KFS organization is a virtual team. For example, the Kuali Board consists of ten persons who come from nine universities and work in different locations and different time zones.

There are three administrative projects in Kuali. Kuali Financial System (KFS) is currently at the final development stage. It expects to expand the KFS into two new systems: the Kuali Coeus (KC), which is the research administration system, and Kuali Student (KS), which is the student service system. The KC and KS projects are scheduled to be released after 2010. There are several projects supporting the three administrative projects including Kuali Rice, Kuali Ole and Kuali Ready.

The initial mission of the Kuali consortium was to develop a baseline system for financial services and has now expanded to other systems such as research administration and student management. In order to ensure flexibility and extensibility, Kuali uses the most up-to-date approaches and technologies such as open source, workflow, and service-oriented architecture. Fig. 2 illustrates the Kuali system architecture and shows how the enterprise service bus is used to support service-oriented architecture.

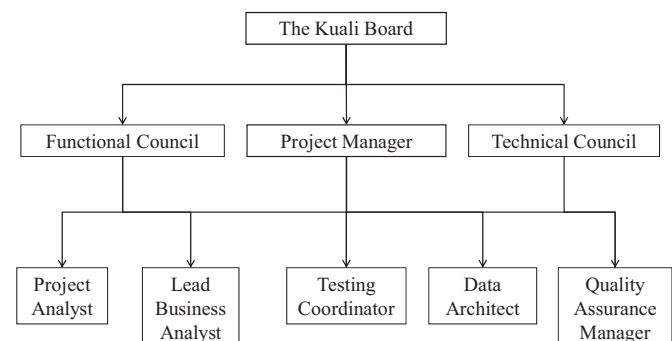


Fig. 1. Kuali organizational chart.

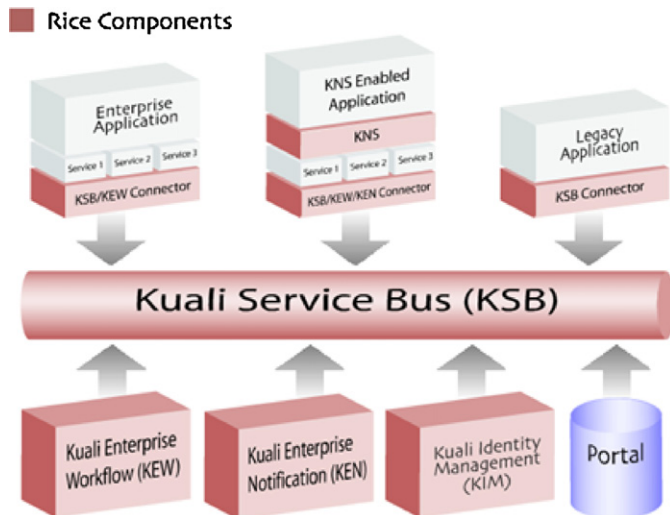


Fig. 2. Kuali rice system architecture (<http://rice.kuali.org/>).

Currently, there are three main application modules including financial services, research administration, and student system.

4.2. The Sakai case—a learning and collaboration system

The Sakai project is a community source software development effort to design, build and deploy a new Collaboration and Learning Environment (CLE) for higher education (www.sakaiproject.org). Sakai's development model is referred to as "community source" because many of the developers creating Sakai are drawn from the "community" of organizations that have adopted and are using Sakai. Sakai is an online collaboration and learning environment that provides a framework and pluggable tool modules for managing, delivering, and assessing student learning. It is designed to bring students and instructors together for knowledge sharing, discussion, and shared learning.

The Sakai Project began in 2004 as a collaboration of four universities (the University of Michigan, Indiana University, Massachusetts Institute of Technology, and Stanford University). The existing home-grown systems of the four universities were unsustainable in the long run although the current needs could be met. The goal of the Sakai project is to pool all the different home-grown systems and create a core common code base that could address the sustainability issue.

The four universities contributed US\$4.4 million to the project. The Sakai project also received a \$2.4 million grant from the Mellon Foundation. Currently the Sakai project is run by the Sakai Foundation, which is a vibrant and growing community of more than 100 members representing universities, colleges, and institutions worldwide.

The organizational structure of the Sakai Foundation is relatively simple compared to that of the Kuali Foundation. The Sakai project is supported by voluntary partner contributions. The Sakai Foundation member organizations elect the ten-member Board of Directors, which provides the strategic leadership for the Sakai Foundation.

Sakai consists of two parts: the Sakai framework and Sakai tools (Farmer and Dolphin, 2005). The framework provides presentation and common services to form a basic system, while tools are designed for specific purposes such as chat room and discussion. Tools can be treated as components and can be plugged-in or removed from Sakai dynamically. Tools can make use of common services such as retrieving current user information. Sakai

Fig. 3. Kuali forms and labels.

provides generic collaboration tools such as announcement, chat room, archive, as well as learning and teaching tools.

5. How Kuali achieves technology flexibility

Kuali achieves technology flexibility via Service Oriented Architecture (SOA) and five levels of system customization. SOA plays an important role in developing new applications and modifying existing applications quickly and easily. The Kuali Rice System is an enterprise middleware suite of integrated products, which enables agile application development and greatly simplifies the deployment and maintenance of all Kuali applications. Fig. 2 shows the key components of the Kuali Rice System. More specifically, Kuali Rice System is designed and built with SOA concepts in mind, including several key services comprised of Kuali Service Bus (KSB), Kuali Nervous System (KNS), Kuali Enterprise Workflow (KEW), Kuali Enterprise Notification (KEN), and Kuali Identity Management (KIM). Kuali Rice System represents a re-usable development framework that advocates a simplified approach to developing business functionality as modular applications.

As a community source project, Kuali must balance varying requirements from many universities. Thus, it is imperative to make the Kuali system flexible. Five levels of customization are enabled in Kuali to achieve flexibility including label customization, modification and addition of document types, workflow customization, code modification, and addition of new modules as discussed in detail next.

5.1. Level 1: label customization

An important aspect of flexibility is to enable the customization of the look and feel of the user interface. Specifically, Kuali allows its adopters to rename any label on all forms (Fig. 3) without the need to write code. Kuali supports label customization by means of its Data Dictionary, which can be specified using XML files. For example, account.xml (Table 2) holds various attributes about a Business Object "Account Number". This customizability allows an institution to change the user interface to fit its unique lingo without changing the lower-level programs.

Table 2
Data dictionary attribute definition.

```
<attribute name="accountNumber" forceUppercase="true">
<label>Account Number</label> <shortLabel>Account</shortLabel>
<maxLength>7</maxLength>
<validationPattern> <alphaNumeric exactLength="7"
allowWhitespace="false"
allowUnderscore="false"/> <validationPattern>
<required>true</required> <control> <text size="10"/> <control>
<summary>Account Number</summary> <description>Identifier for a pool
of funds.</description>
</attribute>
```

Table 3
Kuali document type examples.

Financial Transactions

Budget Adjustment

• Cash Receipt

- Cash Receipt
- Credit Card Receipt
- Advance Deposit
- Cash Management

• Disbursement Voucher

• Non-Check Disbursement

• Pre-Encumbrance

• Distribution of Income and Expense

• Internal Billing

• Service Billing

- Specification
- Issues

5.2. Level 2: modification and addition of document types

Kuali organizes its financial documents into a hierarchy of 139 document types. Table 3 illustrates a number of document types in the financial transactions category. Document types are a critical concept for Kuali since its business processes evolve around the routing of documents. Kuali allows its adopters to modify and add new document types via XML files. As shown in Table 4, the Account Maintenance Document type is defined via XML (only partially shown for simplicity). The components of a document type include business rules, authorization, locking keys, and so on.

Table 4
AccountMaintenanceDocument.xml.

```
<dictionaryEntry>
<maintenanceDocument>
...
<businessRulesClass>org.kuali.module.chart.rules.AccountRule</businessRulesClass>
<documentAuthorizerClass>org.kuali.core.document.MaintenanceDocumentAuthorizerBase</documentAuthorizerClass>
<authorizations>
<authorization action="initiate">
<workgroups> <workgroup>kualiUniversalGroup</workgroup> <workgroups>
</authorization>
</authorizations>
<documentTypeName>KualiAccountMaintenanceDocument</documentTypeName>
<documentTypeCode>ACCT</documentTypeCode>
<label>Account Maintenance Document</label>
<shortLabel>AcctMaintDoc</shortLabel>
<summary>Account maintenance document</summary>
<description>Document used to create or update Account objects</description>
<lockingKeys> <field attributeName="chartOfAccountsCode"/> <field attributeName="accountNumber"/> <lockingKeys>
...
</maintenanceDocument>
</dictionaryEntry>
```

5.3. Level 3: workflow customization

Workflows in Kuali can be customized to suit the needs of different universities. Essentially, Kuali workflow is document-based and allows for modification of document routing without writing any code. As shown in Fig. 4, the Kuali workflow engine specifies the generic actions on financial documents including initial actions, approval actions, workflow processor, and post processor. Specific actions on each financial document are specified within the document type itself such as document initiator, organizational unit, and document approver. The Kuali workflow uses XML-driven routing capabilities, and routing rules can be defined or modified with respect to any document type without requiring a modification to the Kuali source code.

5.4. Level 4: code modification

As an open source system, Kuali is distributed according to the Educational Community License version 1.0, which allows its users to: (1) freely access, install, and run the software for any purpose; (2) modify the original software; (3) redistribute copies of the original or modified programs; and (4) share modifications with the community. It is worth noting that although Kuali is based on many of the principles of open source development, its efforts rely more explicitly on defined roles, responsibilities, and funded commitments by community members than some open source development models (<http://www.kuali.org/faq/23>).

5.5. Level 5: addition of new modules

As discussed in an earlier section, Kuali started as a financial system and has evolved into an umbrella suite of administrative systems for higher education. It accomplishes that function by providing a common architecture that can be extended to incorporate new applications. The Kuali framework improves productivity and consistency across Kuali applications and creates a reusable development environment. The service-centric approach of Kuali makes it possible to develop once and use anywhere and on any platform.

6. How Sakai achieves technology flexibility

The next generation of Sakai platform, named Sakai Open Academic Environment (OAE), is built with technology flexibility in mind. More specifically, Sakai OAE's technology flexibility is classified into two categories, (1) flexibility-for-use, which measures

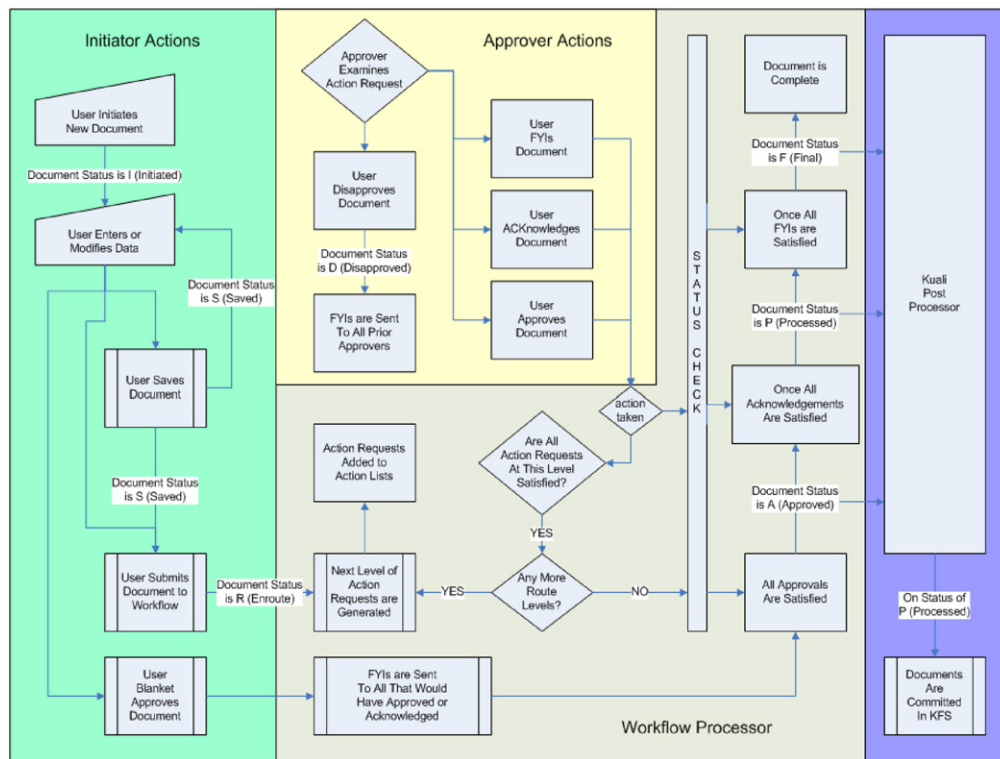


Fig. 4. Kuali workflow.

how easily the system can be configured and personalized to support various learning, teaching, and collaboration needs without programming and (2) flexibility-for-development, which measures how easily the system can be customized and extended to meet new functional requirements.

Sakai OAE's service-oriented system architecture provides the foundation to support the two aforementioned types of technology flexibility. Sakai OAE is built on top of Apache Sling, which is a web framework that embeds an Open Services Gateway initiative (OSGi) framework (Apache Felix) to provide a dynamic runtime environment and uses a Java Content Repository (Apache Jackrabbit) to store and manage content. The OSGi framework enables various code bundles (Sakai services/widgets), such as contact list, recent messages, discussion board, chat room, and blog, to be loaded, unloaded, and reconfigured at runtime (see Fig. 5a). Given the large number of partner institutions involved in the analysis, design and development of the Sakai system, the system requirements are highly diverse and constantly changing.

Although it may not be possible to satisfy every requirement of the community, the component-based flexible architecture enables the Sakai community to focus on a core set of functionalities by developing and maintaining key code bundles centrally while allowing community members to implement specific bundles to satisfy their special needs. When the demand for those specific bundles becomes more dominant, they can be easily incorporated into the Sakai core system and maintained by the community instead of individual institutions. The layout of the widgets can also be easily personalized by dragging and dropping widgets to the desired locations as shown in Fig. 5b, leading to controllable user workspaces with infinite flexibility.

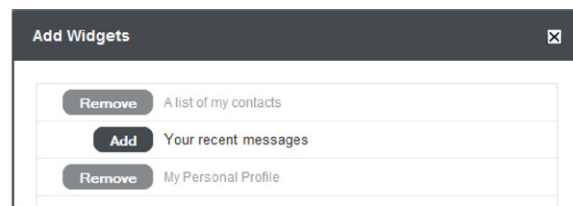
Sakai OAE provides rich feature sets in terms of different widgets to support most common needs for online teaching, learning, and collaboration. In addition, the best practices for internationalization and localization have been followed in Sakai OAE design, so that the user interface can be easily configured to

different languages. Another key service to facilitate flexibility is the integrated workflow and business rule engine named Drools (<http://www.jboss.org/drools>), which allows modeling of complex cross-functional workflows and business rules. In sum, the flexible OSGi framework in Sakai OAE provides great flexibility for use by allowing organizations to choose the best set of features according to their specific needs without programming.

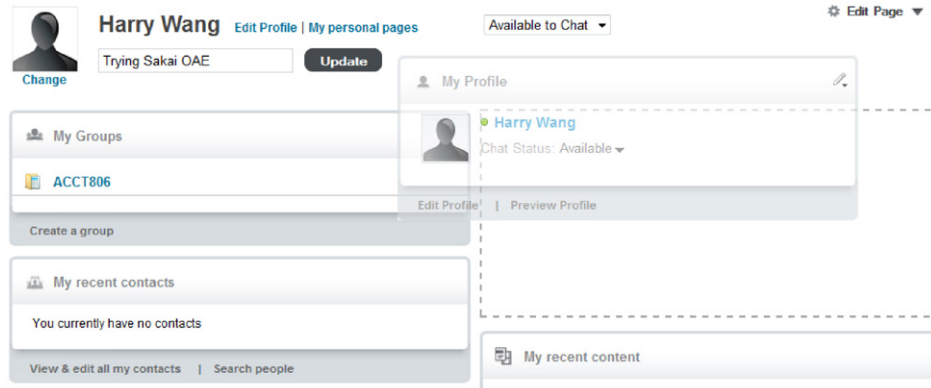
The Java Content Repository (JCR) in Sakai OAE enables flexible content authoring and sharing. Once specific content, such as a page, a document, or a video, is created, the content creator can define fine-grain access control policies to specify how to share the content. In this way, the content can be easily reused and shared across course, college, and institution boundaries. Sakai OAE is Learning Tools Interoperability (LTI) certified, which enables content exchange with other LTI certified systems. For example, courses created in Sakai OAE can be exported and then imported into other LTI certified systems.

Sakai OAE's underlying infrastructure of services links the isolated user experiences in different courses and tools often found in traditional learning management systems. This infrastructure supports two of Sakai OAE's design goals: "Multiple options for and ease of software development and integration" and "Agile Sakai code base built atop other reliable open source projects."

Sakai OAE core developers are employees of different partner institutions and companies, are often geographically distributed and have diverse backgrounds. For such a dynamic development team, Sakai OAE's flexible system architecture enables small groups of team members to work on different Sakai services in parallel without high integration overhead. It also allows an agile development process to be followed to deliver small chunks of functionality frequently. For example, three three-week-long coding "sprints" have been conducted between November 2010 and January 2011 to enhance/release over a dozen existing/new features. The flexibility of Sakai OAE's code base can also be illustrated via its support of many scripting languages, such as Java scripts, PHP, Python, and



(a) Dynamic Loading/Unloading of Sakai Widgets



(b) Dynamic Layout Configuration of Sakai Widgets

Fig. 5. Sakai widgets as a means of technology flexibility.

Ruby. This flexibility enables web developers with different expertise to join and contribute to the community.

7. Principles of technology flexibility in community source

Technology flexibility in community source is a multi-dimensional subject matter, and understanding the associated issues should be useful to understanding principles of software engineering. Based on our studies of the Kuali and Sakai cases, we find the following principles:

- **Principle 1:** Community source focuses on enterprise applications rather than generic system-level software, and thus community source requires more technology flexibility. As illustrated by the Kuali and Sakai cases, each participating organization is interested in developing software tools that solve their business problems. In the case of Kuali, an application has to do with automating a particular business process as in financial applications or research administration, and in the case of Sakai, an application is about some aspect of the course administration process. This is very different from system-level software such as Linux since an organization would not be interested in devoting resources if the software is as generic as the Linux operating system. As such, this principle shows that community source requires more technology flexibility than conventional open source.
- **Principle 2:** User requirements in community source are very complex and volatile, thus requiring more technology flexibility. In the Kuali and Sakai cases, each community consists of tens or hundreds of participating organizations, each of which contributes human and financial resources over a long period of time as the community source is being developed and deployed. Given that business applications vary from organization to organization due to variations in business processes that are governed by organizational orientations and regulations, the overall system requirements of all participating organizations are very complex. Further, partner organizations change significantly over the

course of system development, causing user requirements to vary. As such, the underlying technology and system infrastructure must be flexible and customizable.

- **Principle 3:** The volatile developer composition in community source requires even more technology flexibility. We found in our research with Kuali that developer instability is another reason for requiring more technology flexibility (Liu et al., 2010). Three facts contribute to the problem of developers' instability in community source. First, the turn-over rate of IS staff in the home institution itself is high. Second, the development partners might assign the developer to other projects and replace him with a new developer. Lastly, when the community source organization is growing, new partners will join the community source project and a few old partners might drop out. Therefore, the community source needs to keep on training new developers to make sure that they get familiar with the work very quickly. The XML-based technology in Kuali and support of multiple popular programming languages in Sakai make the training easier to new developers. In a way, this is a feature of technology flexibility from the developer perspective.
- **Principle 4:** Technology flexibility in community source is needed to support more system transparency. The feature of collaboration states that the Kuali community's success comes from the members' ability to collaborate, which entails a number of underlying values including listening, introspection, objectivity, and thoughtful communication. The feature of transparency means that Kuali's communication and collaboration infrastructure is open to the public to a large degree. Although certain information may be protected for various reasons, Kuali strongly values making as much information publicly accessible as possible. This high level of transparency can only be supported when user requirements are well-specified and their changes are done systematically, thus requiring more technology flexibility.
- **Principle 5:** Technology flexibility in community source is achieved via modularization and customization. Community source requires highly flexible software in order to meet the needs of the community. Software modularization and

customization enable better scalability, software distribution, and reusability. The feature of scalability means that Kuali and Sakai produce software that can serve both small and large user groups. The feature of distributed software states that software architecture of Kuali and Sakai is guided by the principle of distribution, resulting in loosely coupled components that allow for flexibility and a rapid response to changes in business needs. The feature of reusability indicates that the Kuali and Sakai communities work hard to identify common needs between different applications, and produce assets that can be easily reused throughout the communities.

- **Principle 6:** Technology flexibility in community source is further enabled by means of SOA, document-centric workflow, and rule-based modeling. SOA is one of the key components of community source system development, which enables the Kuali adopters to extend existing system functionality by adding new application modules in a “plug-and-play” fashion. For instance, the authentication system for the Kuali Student system is designed as services based on service contracts, allowing the leverage of any contract-aware authentication system. Similarly, Sakai also leverages Apache Sling to achieve service-oriented system architecture. The workflows and business rules supporting various system functions should not be hardcoded into the program. Instead, workflow and business rule engines should be leveraged to support easy workflow and business rule design, customization, and execution.

8. Conclusions

Our study of the community source phenomenon in the cases of Kuali and Sakai reveals that recent innovations in software technologies include novel software engineering techniques such as SOA, document-based workflow, rule-based policies, and XML-based programming. A common thread of technical innovation among these novel techniques is to enable high technology flexibility. This issue of flexibility is especially relevant to community source since it requires robust applications for an evolving and complex community of development partners. In this paper, our study of two community source cases, i.e., Kuali and Sakai, shows that the technology flexibility principles depend on an interwoven relationship between technological constraints and organizational factors.

We believe that the collection of principles generated from the two real world cases in this paper is unique to community source because of several reasons. First, community source is a special form of open source that focuses on enterprise applications rather than generic systems. This makes the software more specific to the institutional needs. Second, the community source is also a virtual organization consisting of varying partner institutions. This organizational volatility is not found in other forms of software development. Third, the first two reasons lead to special solutions to technology flexibility found in the paper. This finding is quite insightful since few similar findings are found in the literature. All in all, this unique set of observed principles should lead to more exciting research in technology flexibility, a very important but difficult subject to study in software development.

We hope that our case study results have instructional value in other forms of software development although they are more pronounced in community source that requires more flexibility due to their complex requirements from multiple partner developers.

There are two limitations in this paper. First, the cases were generated from the higher education sector. Because universities are quite different from commercial companies, the research result may not be directly applicable to other industries. Accordingly, one

future research direction is to study community source projects outside the higher education sector as they become available. Second, the six principles are derived based on observing the two community source cases but require more empirical validation. In future research, we will monitor the development and deployment of Kuali and Sakai systems presented in this paper. A survey as well as interviews has been planned among partner institutions in Kuali. Empirical data will be collected to validate the principles.

References

- Adler, P., 1998. Automation, skill and the future of capitalism. *Berkeley Journal of Sociology*.
- Bieberstein, N., Bose, S., Fiamante, M., Jones, K., Shah, R., 2006. Service-Oriented Architecture Compass.
- Curtis, B., Krasner, H., Iscoe, N., 1988. A field study of the software design process for large systems. *Communications of the ACM* 31 (November (11)), 1268–1287.
- Davenport, T., Linder, J., 1994. Information management infrastructure: the new competitive weapon. In: *Proceedings of the Twenty-Seventh Hawaii International Conference on System Sciences*, vol. IV.
- De Leeuw, A.C.J., Volberda, H.W., 1996. On the concept of flexibility: a dual control perspective. *Omega: International Journal of Management Science* 24 (2), 121–129.
- Deiters, W., Goesmann, T., Löffler, T., 2000. Flexibility in workflow management: dimensions and solutions.
- Ducheneaneut, N., 2005. Socialization in an open source software community: a socio-technical analysis. *Computer Supported Cooperative Work* 14, 323–368.
- Duncan, N.B., 1995. Capturing flexibility of information technology infrastructure: a study of resource characteristics and their measure. *Journal of Management Information Systems* 12 (2), 37–57.
- Eisenhardt, K., 1989. Building theories from case study research. *Academy of Management Review* 14 (4), 532–550.
- Evans, J.S., 1991. Strategic flexibility for high technology maneuvers: a conceptual framework. *Journal of Management Studies* 28 (1), 69–89.
- Farmer, J., Dolphin, I., 2005. Sakai: eLearning and more. In: *Proceedings of the 11th European University Information Systems*, Manchester, UK.
- Fitzgerald, B., 2006. The transformation of open source software. *MIS Quarterly* 30 (September (3)), 587–598.
- Gupta, Y.P., Somers, T.M., 1992. The measurement of manufacturing flexibility. *European Journal of Operational Research* 60 (2), 166–182.
- Keng, S., Ee-Peng, L., Xixing, S., 2001. Mobile commerce: promises, challenges, and research agenda. *Journal of Database Management* 12 (3), 4.
- Kettinger, W.J., Marchand, D.A., Davis, J.M., 2010. Designing enterprise IT architectures to optimize flexibility and standardization in global business. *MIS Quarterly Executive* 9 (June (2)).
- Kraut, R.E., Streeter, L., 1995. Coordination in software development. *Communications of the ACM* 38 (March (3)), 69–81.
- Liu, M., Tu, Q., 2011. Community-based open source: the phenomenon and research opportunities. In: *Proceedings of the 13th International Conference on Enterprise Information Systems*.
- Liu, M., Wang, H.J., Zhao, J.L., 2007. Achieving flexibility via service-centric community source: the case of Kuali. In: *Proceedings of the Americas Conference on Information Systems*, August 9–12, 2007.
- Liu, M., Wu, X., Zhao, J.L., Zhu, L., 2010. Outsourcing of community source: identifying motivations and benefits. *Journal of Global Information Management* 18 (4).
- Mason, S.P., 1986. *Valuing Financial Flexibility*. National Bureau of Economic Research Project Report Series. University of Chicago Press, Chicago/London.
- Nelson, K.M., Nelson, H.J., Ghods, M., 1997. Technology flexibility: conceptualization, validation, and measurement. In: *Proceedings of the 30th Hawaii International Conference on System Sciences*.
- Ocker, R.J., Fjermestad, J., 2000. High versus low performing virtual design teams: a preliminary analysis of communication. In: *Proceedings of the 33rd Annual Hawaii International Conference on System Sciences*, vol. 1, p. 9.
- Horn, P., 2001. *Autonomic Computing: IBM's Perspective on the State of Information Technology*.
- Perry, R., Quirk, K., 2007. An evaluation of build versus buy for portal solutions. IDC White Paper. Sponsored by IBM.
- Ramasesh, R.V., Jayakumar, M.D., 1991. Measurement of manufacturing flexibility: a value based approach. *Journal of Operations Management* 10 (4), 446–468.
- Stewart, K.J., Gosain, S., 2006. The impact of ideology effectiveness in open source software development teams. *MIS Quarterly* 30 (June (2)), 291–314.
- Wang, H.J., Blue, J., Plourde, M., 2010. Community source software in higher education. *IT Professional* 12 (6), 31–37.
- Wheeler, B., 2004. The open source parade. *EDUCAUSE Review*, 68–69.
- Wheeler, B., 2007. Open source 2010: reflections on 2007. *EDUCAUSE Review* 42 (1), 49–52.
- Whiting, R., 2003. A win-win combination? *Informationweek* 931, 57–58.
- Yin, R., 1994. *Case Study Research. Design and Methods*. Sage Publications, Thousand Oaks.
- Zeng, D.D., Zhao, J.L., 2002. Achieving software flexibility via intelligent workflow techniques. In: *Proceedings of the 35th Hawaii International Conference on System Sciences*, January 7–10, 2002.